A COURSE FOR COMPLETE BEGINNERS · NO PRIOR KNOWLEDGE NEEDED

Building a website from A to Z

Understand how a real website is built, put online and managed day to day — by opening, file by file, the one of **The Evergreen Project**.

Website: dimogdor.github.io/Evergreen · Source code: github.com/Dimogdor/Evergreen

HOW TO READ THIS COURSE

Each chapter explains one idea simply, then shows **where to see it in the site** (green "Look" box) and sometimes suggests a small **try-it** (orange box). The blue "Say it" boxes are key sentences for explaining out loud. No need to remember everything: the goal is to understand *who does what*.

Contents

1. How a website works
2. The three languages of the web: HTML, CSS, JavaScript
3. The toolbox
4. The GitHub repository: the project folder
5. Astro: the site “generator”
6. Keeping content apart from code (YAML)
7. Design in practice: from idea to screen
8. Layout and responsive design (phone, tablet, computer)
9. Photos and videos
10. A website in 8 languages
11. The features, one by one
12. The admin area (no code)
13. The small server: the Worker
14. Being found on Google (SEO)
15. Accessibility: a website for everyone
16. Security
17. Speed (performance)
18. Testing the website
19. Going live: hosting, domain, DNS
20. The life of a change
21. Where to start learning
22. Glossary

TO SAY AS AN INTRODUCTION

“A website is just text files that a browser knows how to read. Everything else — the tools, the services — is there to write those files faster, make them beautiful, and put them online.”

1 · How a website works

When you type an address into a browser (Chrome, Safari, Firefox…), three things happen:

Browser → asks → Server (a computer that is always on) → sends back files → The browser displays them

1. **The address** (URL), e.g. `https://dimogdor.github.io/Evergreen/en/contact/`, says which server to contact and which page to ask for.
2. **The server** sends back files: an **HTML** page, **CSS** style sheets, **JavaScript** scripts, images.
3. **The browser** puts everything together and shows the page.

Static or dynamic website?

	Static website (our case)	Dynamic website (e.g. WordPress)
Principle	Pages are built **in advance**, once, then simply sent.	Each page is built **on every visit** by a program and a database.
Speed	Very fast	Slower
Security	Very few possible weak spots	Frequent security updates
Cost	Free hosting	Usually a paid server

The Evergreen website is **static**: after every change, a tool builds all the pages (171 of them!) in a few seconds, then they are placed with the host. We will see later how we still get an admin area and forms.

> **LOOK**
> Open the website, then right-click → **"View page source"**: this is the HTML the server sent. Then right-click → **"Inspect"**: these are the browser's **developer tools**, the best friend of anyone who builds websites.

2 · The three languages of the web

Every website in the world relies on three languages, each with a precise role. A simple picture: a house.

Language	Role	In the house
HTML	The **structure** and the content: headings, paragraphs, images, links, forms.	The walls and the rooms
CSS	The **look**: colours, fonts, sizes, spacing, layout.	The paint and the decoration
JavaScript	The **behaviour**: what reacts when you click, what changes without reloading.	The electricity, the switches

HTML: “tags”

```
<h1>Contact</h1>                      ← a big heading
<p>Write to us, we answer quickly.</p>  ← a paragraph
<a href="mailto:evergreenecovillage@gmail.com">Write to us</a>  ← a link
<img src="tent.webp" alt="A tent in the forest">          ← an image + its description
```

CSS: “rules”

```
h1 { font-family: 'EB Garamond'; color: #22402a; }   ← every big heading in dark green
.btn { border-radius: 999px; padding: 0.85em 1.4em; } ← rounded buttons
```

JavaScript: “instructions”

```
button.addEventListener('click', () => {               ← when the button is clicked…
  navigator.clipboard.writeText('GR7701103910000039100269800');  ← …copy the IBAN
});
```

LOOK IN THE REPOSITORY

- CSS: `src/styles/global.css` — all the site's styling in a single file.
- JavaScript: at the bottom of `src/pages/[lang]/[page].astro`, the IBAN “Copy” button (the `<script>` lines).
- HTML: everywhere in the `.astro` files (see chapter 5).

And TypeScript?

It is JavaScript with “labels” that state the type of each piece of data (text, number…). It prevents mistakes. The project's `.ts` files (e.g. `src/calendar.ts`) are written in it.

SAY IT

“Start with HTML and CSS: in two weeks you can make a real page. JavaScript comes later, when you want the page to react.”

3 · The toolbox

Tool	What it is for	Free?
A code editor: VS Code	Writing and organising the files, with colours and suggestions.	Yes
A browser + its developer tools (F12)	Seeing the result, inspecting, testing on a “fake” phone.	Yes
The terminal	Typing commands (start the site, install tools).	Yes
Node.js and **npm**	Node runs JavaScript on the computer; npm installs “packages” (tools written by others).	Yes
Git	Keeping the history of every change (like “Undo”, but forever).	Yes
GitHub	Storing the project online, working together, triggering automatic publishing.	Yes
Figma (optional)	Drawing mock-ups before coding.	Yes (basic plan)

The project's three commands

```
npm install     ← downloads the tools listed in package.json (only once)
npm run dev     ← runs the site on the computer: http://localhost:4321 (updates at every save)
npm run build   ← builds the final version in the dist/ folder
```

LOOK

`package.json` : the project's “ID card”. It lists the commands (`scripts`) and the packages used (`dependencies`): **astro** (the generator), **@astrojs/sitemap** (site map for Google), **marked** (turns simple text into HTML), **yaml** (reads the content files), and two fonts (**Inter** and **EB Garamond**). `.node-version` : the Node version to use (22).

Git in 5 words

- **Repository**: the project folder with its whole history.
- **Commit**: a “snapshot” of the files at a moment, with a message (“Inhabitants: new photo of our dogs”).
- **Branch**: a parallel working copy to try things without breaking the official version (`main`).
- **Pull request**: the request to merge a branch into `main` , with review.
- **Push / pull**: send / fetch commits to / from GitHub.

TRY IT

On github.com/Dimogdor/Evergreen, click **“Commits”**: you see every change made to the website, who made it and when, and you can display exactly which lines changed.

4 · The repository: the project map

Here is how the folders are organised. Tidying well is half of the work.

```
Evergreen/
├── package.json            ← ID card, commands, tools
├── astro.config.mjs        ← generator settings (site address, security)
├── README.md               ← the project's manual
├── docs/GUIDE-ADMIN.md     ← the manual for the admins
├── public/                 ← files copied as they are
│   ├── admin/              ← the admin area (config.yml, index.html)
│   ├── uploads/videos/     ← videos
│   └── _headers            ← security/cache rules for the host
├── src/                    ← the heart of the site
│   ├── pages/              ← one page = one file (chapter 5)
│   ├── layouts/Base.astro  ← the shared frame: header, menu, footer
│   ├── components/         ← reusable bricks (image, event list…)
│   ├── styles/global.css   ← the whole design
│   ├── content/            ← ALL the site's text, in YAML files (chapter 6)
│   ├── assets/uploads/     ← original photos (optimised automatically)
│   ├── lib.ts              ← small helper functions (read content, links…)
│   ├── i18n.ts             ← the languages
│   └── calendar.ts         ← dates, time zones, calendar files
├── worker/                 ← the small server (admin sign-in, stories, writing help)
├── tests/                  ← automatic checks
└── .github/workflows/      ← GitHub robots: publishing, video compression
```

SAY IT

“The code (pages, design) and the content (texts, photos) live in separate folders. The people running the place only touch the content, through the admin; the code hardly ever changes.”

5 · Astro: the site “generator”

Writing 171 HTML pages by hand would be impossible (8 languages × about twenty pages). So we use a **static site generator**: we write **one template** per kind of page, and the tool fills it with each piece of content, in each language. Here it is **Astro** (astro.build).

An .astro file = two parts

```
---
// Part 1, between the dashes: JavaScript that prepares the data (runs while the site is built)
const page = await one('pages', 'contact', lang);
---
<!-- Part 2: HTML, into which the data is inserted between curly braces -->
<h1>{page.title}</h1>
<p class="lead">{page.intro}</p>
```

Pages = addresses

In `src/pages/`, the file name becomes the address. Square brackets `[ ]` mean “variable”:

File	Addresses produced
`src/pages/[lang]/index.astro`	/el/, /en/, /fr/... (the 8 home pages)
`src/pages/[lang]/[page].astro`	/en/contact/, /en/place/, /en/support/... (all the text pages)
`src/pages/[lang]/activities/[slug].astro`	/en/activities/retreats/... (one page per activity)
`src/pages/[lang]/calendar.astro`	/en/calendar/
`src/pages/[lang]/gallery.astro`, `stories.astro`	the gallery, the stories
`src/pages/calendar.ics.ts`	/calendar.ics: a calendar file, not a page!

The layout: the shared frame

`src/layouts/Base.astro` holds what is the same on every page: the invisible `<head>` (tab title, description for Google, icon), the header with the logo and menu, the language switcher, the footer with the contacts. Each page “fits” into it with `<Base> … </Base>`.

Components: LEGO bricks

The simplest one to understand: open `src/components/Story.astro` (12 lines) — a photo, a quote (`<blockquote>`), the name and the country.

Component	What it does
`src/components/Img.astro`	Shows any photo, optimised automatically (chapter 9).
`Blocks.astro`	The “heading + text + photo” sections that alternate left/right.
`EventList.astro`	The list of events with dates, prices, “Add to calendar” button.
`Story.astro`	A story card.
`Video.astro`	A YouTube/Vimeo video that only loads when clicked.

6 · Keeping content apart from code

All the texts, dates, prices and photo paths are stored in `src/content/`, in **YAML** format: a very readable text format made of “key: value” lines and indentation.

> **LOOK: AN EVENT**
>
> `src/content/events/essence-of-yoga-2026.yml`

```yaml
el:                                  ← the Greek version (main language)
  start: '2026-11-28'                ← start date
  end: '2026-12-05'                  ← end date
  category: retreat
  image: /src/assets/uploads/yin-shala.webp
  link: https://bookretreats.com/r/8-day-essence-of-yoga-…
  title: Essence of Yoga — χειμερινό εντατικό …
  price: 380 € early bird (κράτηση έως 28/10/2026) · 450 € κανονική τιμή
en:                                  ← in English: only what gets translated
  title: Essence of Yoga — winter immersion with Panos Katsandris
  price: €380 early bird (book by 28/10/2026) · €450 regular price
```

Folder	Content
`settings/site.yml`	Name, e-mail, phone, social networks, IBAN, site colours
`settings/ui.yml`	The small interface texts (menu, buttons) in the 8 languages
`pages/`	The text of each page (home, the place, way of life…)
`programs/`	The activities (volunteering, retreats…)
`events/`	The calendar: one file per event
`inhabitants/`	The inhabitants: Katerina, Kostas, the dogs, the cats, the hens…
`albums/`, `testimonials/`, `extra/`	Gallery, stories, pages created in the admin

How does the code read this content? `src/content.config.ts` declares the folders, and the `list()` function in `src/lib.ts` reads them and picks the right language.

> **SAY IT**
>
> “No database: the content is plain text files sorted into folders. You can read them, back them up, go back in time. The admin just edits these files for us.”

7 · Design in practice: from idea to screen

Step 1 — Gather inspiration (a "moodboard")

Before coding, look at websites you like and write down *why*. For Evergreen: kaleidoscope.media (elegant big headings, lots of space, numbered sections "01 —") and evergreenyogastudios.com (the Evergreen family identity). Also collect the raw material: logo, recent photos, tone of the texts.

Step 2 — Choose a palette

4 to 6 colours are enough, taken from the subject: here cream paper, moss, deep forest, terracotta. Check the **contrast** (text readable on its background) with a tool such as *webaim.org/resources/contrastchecker*.

```
:root {                       ← src/styles/global.css, at the very top
  --paper: #f4f0e6;           cream background
  --ink: #17231a;             near-black text, tinted green
  --moss: #3d673f;            moss green
  --moss-deep: #22402a;       forest green (ESC banner, footer)
  --clay: #b0622a;            terracotta (accents)
}
```

These "variables" (`--paper`…) are reused everywhere; changing one line changes the whole site. It can even be changed from the admin (Settings → Colours).

Step 3 — Choose two fonts

A **serif font** for headings (EB Garamond: warm, "bookish") and a **simple font** for body text (Inter: very readable on screen). They come from Google Fonts but are **installed inside the project** through the `@fontsource-variable/…` packages: no call to Google, faster and kinder to privacy.

Step 4 — A scale of spacing and sizes

Instead of fixed sizes, use `clamp(min, ideal, max)`: the heading grows with the screen without ever getting too small or too big. Example: `--gutter: clamp(16px, 4vw, 56px)` = the side margin.

Step 5 — Consistent "visual components"

Round buttons in spaced capitals (`.btn`), small labels (`.label`), photo + title cards (`.card`), numbered horizontal rules (`.rule`). Once defined, they are reused: the site looks "designed", not patched together.

Step 6 — Mock-up (optional) then code

In Figma, draw the home page for computer and phone. Then turn it into HTML (structure) and CSS (style) — and fine-tune directly in the browser with the developer tools.

> **TRY IT**
>
> Open the site, F12 → "Elements" tab, click `<html>` and change `--moss-deep` to `#5a2a6e`: the whole site changes colour live (nothing breaks, just reload).

8 · Layout and responsive design

The same site must look good on a phone (390 px wide) and on a big screen (1440 px and more). This is **responsive design**.

The two layout tools of modern CSS

Tool	For what	Example in the site
Flexbox	Lining elements up in a row (or a column).	The header: logo on the left, menu on the right (`.site-header`).
Grid	A grid of rows and columns.	The activity cards, the inhabitants (`.grid.cols-3` , `.cols-4`).

```
.cols-3 { grid-template-columns: repeat(auto-fill, minmax(min(100%, 340px), 1fr)); }
← “as many columns of at least 340 px as the width allows”: 3 on a computer, 1 on a phone, nothing else needed

@media (max-width: 760px) {                          ← “if the screen is narrower than 760 px…”
  .event { grid-template-columns: 1fr; }              ← …events become a single column
  .event .thumb { order: -1; }                        ← …and the photo moves above the text
}
```

Media queries (`@media`) are “if… then…” rules depending on the screen size. But the “self-adapting” grid of the first example often makes them unnecessary: the footer works the same way (`minmax(180px, 1fr)`).

The menu on a phone

On a small screen the menu folds behind a “Menu” button. The code is in `src/layouts/Base.astro` (the `.menu-btn` button) and the style in `global.css` .

Details that look “pro”

- `text-wrap: balance` : two-line headings get lines of equal length.
- `@media (prefers-reduced-motion)` : animations are turned off for people who are bothered by them.
- A maximum width (`--max: 1440px`) so text does not stretch across a giant screen.

> **TRY IT**
>
> F12 → phone/tablet icon (“device mode”) → choose “iPhone”: watch the site rearrange itself. Widen the window little by little and see when the columns change.

9 · Photos and videos

Images are often 80% of a page's weight. A phone photo weighs 4 to 8 MB; on the site, the same photo weighs 50 to 200 KB. How?

Three automatic optimisations

1. **When uploaded in the admin**: the photo is reduced to 2000 px maximum and converted to **WebP**, a lighter format than JPEG (the `transformations` setting in `src/admin/config.yml`).
2. **When the site is built**: Astro makes **5 sizes** of every photo (480, 800, 1200, 1600, 1920 px). File: `src/components/Img.astro`.
3. **In the browser**: thanks to the `srcset` and `sizes` attributes, a phone downloads the small version, a big screen the large one. And `loading="lazy"`: a photo is only downloaded when you scroll near it.

Alternative text

Every image has an `alt`: a description read aloud to blind people and used by Google. E.g. on the home page: “Final relaxation in the shala of the Evergreen Project” (editable in the admin).

Videos

- **YouTube / Vimeo** (recommended): paste the link; the `Video.astro` component only shows a thumbnail and loads the player when clicked (the “youtube-nocookie” version, without cookies) → a fast, respectful page.
- **MP4 file**: a GitHub robot (`.github/workflows/compress-videos.yml`) recompresses it with the **ffmpeg** tool (H.264 format, 1280 px max).

The full-screen viewer

In `src/pages/[lang]/gallery.astro`, clicking a photo opens it large in a native HTML `<dialog>` element — no external library: the browser already handles the window, the Esc key and the focus.

10 · A website in 8 languages

Greek (default language), English, French, Spanish, Italian, Russian, German, Portuguese. Three mechanisms:

1. **Addresses** start with the language: `/el/`, `/en/`… The `[lang]` folder in `src/pages/` builds every page in every language. The list of languages is in `src/i18n.ts`.
2. **The content**: each YAML file has one part per language (chapter 6).
3. **The small texts** (“Menu”, “Read more”, “Add to calendar”…): `src/content/settings/ui.yml`, 96 phrases × 8 languages.

The “fallback”

If a phrase is not translated in a language, the site shows the English, otherwise the Greek. There is never a gap. The code: the `localize` function in `src/lib.ts`.

Telling Google that translations exist

In the `<head>` of every page (`Base.astro`), `<link rel="alternate" hreflang="fr">` tags point to the versions in other languages: a French visitor lands on the French page.

> **LOOK**
>
> On the site, the language switcher at the top right: it keeps the same page and only changes the prefix (`/en/contact/` → `/de/contact/`).

11 · The features, one by one

For each feature: what it does, the tool or service used, and the file to open.

Feature	How it is done	Where to look
Monthly calendar	A small JavaScript script draws the month and colours the busy days; past events disappear on their own (compared with today's date).	`src/pages/[lang]/calendar.astro`, `src/components/EventList.astro`
"Add to calendar" (Google)	A simple link to Google Calendar holding the title, the dates and the "Europe/Athens" time zone: Google converts to the visitor's time.	`src/calendar.ts` (function `googleUrl`)
Apple / Outlook (.ics)	An `.ics` file: a standard text format (RFC 5545) understood by every calendar. Times are converted to universal time with the `Intl` tool built into JavaScript.	`src/calendar.ts` (function `ics`), `src/pages/[lang]/events/[id].ics.ts`
Subscribe to the calendar	A `/calendar.ics` file with every event; the visitor's calendar re-reads it regularly.	`src/pages/calendar.ics.ts`
Map	An **OpenStreetMap** map (free and open) inside an `<iframe>`, plus an "Open in maps" link to Google Maps.	`src/pages/[lang]/[page].astro` (contact part)
Contact form	The **Web3Forms** service: the form sends the fields to their server, which forwards them by e-mail. An invisible box traps robots (a "honeypot").	same file, `<form action="https://api.web3forms.com/submit">`
Donation / IBAN	The IBAN shown in groups of 4 + a button that copies it (the browser's **Clipboard** API).	same file, `donate` block + `<script>` at the bottom
Stories	Form → small server (Worker, chapter 13) → saved "unpublished" → an admin approves.	`src/pages/[lang]/stories.astro`, `worker/index.js`
Gallery	Albums in a CSS "mosaic", `<dialog>` viewer.	`src/pages/[lang]/gallery.astro`
Announcement banner	If the admin's "Ανακοίνωση" field is filled in, an orange strip appears at the top of every page.	`Base.astro` (`site.announcement`)
Link to the yoga studios	An external link (no duplication: classes and prices stay on their own site).	`src/content/settings/site.yml`
Text formatting	Long texts are written in **Markdown** (`**bold**`, `- list`, `[link](address)`), turned into HTML by the **marked** package.	`src/lib.ts` (function `md`)

12 · The admin area (no code)

Katerina and Kostas are not going to edit YAML files. So they get an **interface** at the address `/admin/`: forms, buttons, drag-and-drop for photos. It is a **CMS** (Content Management System).

The chosen CMS: Sveltia CMS

- Free and open source, runs entirely in the browser.
- It reads and writes the repository's YAML files directly: **no database**.
- Handles the 8 languages side by side, compresses photos when they are uploaded.

How it is set up

Only two files:

- `public/admin/index.html`: loads the Sveltia program (pinned version + security fingerprint, see chapter 16).
- `src/admin/config.yml`: describes **every form**. For each field: its name, its label, its type ("widget").

```
- { name: start, label: Έναρξη · Start date, widget: datetime, type: date }   ← a calendar to pick the date
- { name: price, label: Τιμή · Price, widget: string, i18n: true }          ← a text field, translatable
- { name: image, label: Φωτογραφία · Photo, widget: image }                 ← a photo upload button
- { name: published, label: Published, widget: boolean }                    ← a checkbox
```

What happens when you click "Save"

Admin: Save → The YAML file changes in the repository → A robot rebuilds the site → Online ~2 min later

Every save is a "commit": you can always go back to an earlier version.

Writing help

A **"✓ Correct"** button appears next to the field being edited: it suggests a corrected version (spelling, grammar, punctuation), to accept or not. And on every save, new or changed Greek texts are **translated automatically** into the 7 other languages. Both go through the Worker (chapter 13), which calls an automatic correction and translation service. Code: `public/admin/index.html` (the button) and `worker/ai.js`.

> **LOOK**
>
> `docs/GUIDE-ADMIN.md`: the manual written for the admins (Greek + English). A good example of documentation for non-technical people.

13 · The small server: the Worker

A static site cannot keep a secret or receive data (no program runs on the server). So, for a few needs, we add a **tiny server**, free, at **Cloudflare**: a **Worker** (a JavaScript function that runs when it is called).

Need	What the Worker does
Admin sign-in	Shows an e-mail + password form, checks it, then issues a session “badge” (valid 30 days), signed with a secret key (an HMAC signature) so it cannot be forged.
Admin saves	Relays the admin's requests to the storage, adding the real access key that only the Worker knows.
Writing help	Corrects a text on request; translates every save into the 7 languages before storing it.
Stories	Receives the form, filters spam, checks that the photo really is an image, saves the story “unpublished” and notifies the team.

The “secrets” (admin passwords, access key) are kept at Cloudflare, encrypted — **never** in the repository's code.

> **LOOK**
>
> `worker/index.js` (well commented, split into sections: sign-in, relay, stories) · `worker/ai.js` (writing help) · `worker/wrangler.toml` (settings) · `worker/README.md` (step-by-step installation). The **wrangler** tool (`npx wrangler deploy`) sends the Worker to Cloudflare.

API: the word to know

An **API** is a “door” that a program opens to other programs: you send a request to an address, you get an answer. The site uses several:

API	Used for
GitHub API	Reading / writing the content files (through the Worker)
Translation / correction service	Translating and correcting the admin's texts (through the Worker)
Web3Forms	Sending the contact form by e-mail
Cloudflare Turnstile (optional)	Anti-robot check, a respectful alternative to “CAPTCHAs”
Browser APIs	Clipboard (copy the IBAN), `Intl` (dates and time zones), `<dialog>`
Google Calendar (link)	Adding an event to one's calendar

14 · Being found on Google (SEO)

SEO (search engine optimisation) means helping search engines understand and present the site well.

Element	Role	Where
`<title>` and `<meta name="description">`	The blue title and the two lines of text in Google results.	`Base.astro`
Open Graph (`og:image`…)	The preview (photo + title) when the link is shared on Facebook, WhatsApp…	`Base.astro`
Site map (`sitemap-index.xml`)	The list of every page for Google; generated by **@astrojs/sitemap**.	`astro.config.mjs`
`robots.txt`	Tells robots what they may visit (not the admin).	`src/pages/robots.txt.ts`
Structured data (JSON-LD)	A machine-readable card: “this is an NGO, here is its address, its phone” (the **schema.org** vocabulary).	`Base.astro` (`ld`)
`hreflang`	The translations of each page.	`Base.astro`
Readable addresses	`/en/contact/` rather than `/page?id=37`.	file names in `src/pages/`

TRY IT

Paste the site's address into *opengraph.xyz* to see the sharing preview, or search for Google's “Rich Results Test” to see the JSON-LD card.

15 · Accessibility: a website for everyone

About one person in five has a disability (sight, mobility, hearing, attention…). An accessible site is also nicer for everyone, and better ranked by Google.

- **Alternative texts** for images (`alt`).
- Sufficient **contrast**: light grey text on a cream background is unreadable for many. The footer credit was darkened for that reason.
- **Headings in order** (`h1` then `h2`…): screen readers use them as a table of contents.
- A hidden **“Skip to content” link**, visible from the keyboard (`.skip` in `Base.astro`): no need to go through the whole menu.
- **Keyboard navigation**: a visible outline on the selected element (`:focus-visible` in `global.css`).
- **Clear names** for buttons and links without text (`aria-label`): e.g. the photo links on the home page.
- Respecting the phone's “reduce motion” setting.

> **TRY IT**
>
> On the site, press the **Tab** key several times: you must always see where you are. Then F12 → **Lighthouse** tab → “Accessibility” → Analyze.

16 · Security

Risk	Protection in this site	Where
An attacker injecting a script into a page	**CSP** (Content-Security-Policy): a whitelist of what the page may load; the browser blocks everything else.	`astro.config.mjs` (`security.csp`)
A third-party service (the CMS) being changed behind our back	**Pinned version + SRI**: a fingerprint of the file; if a single byte changes, the browser refuses to run it.	`public/admin/index.html` (`integrity="sha384-…"`)
Leaked passwords / keys	Secrets only at Cloudflare, never in the repository; local secret files ignored by Git.	`.gitignore`, `worker/README.md`
Password guessing in bursts, spam	Limit of 3 attempts / minute / visitor, trap field, minimum delay, optional Turnstile.	`worker/index.js`, `wrangler.toml`
Fake files being uploaded	Checking the first bytes (the “signature”): a JPEG always starts with `FF D8 FF`.	`worker/index.js` (`looksLike`)
Eavesdropping on the connection	HTTPS everywhere (the padlock).	host
The site shown inside a trap frame	The `X-Frame-Options` header.	`public/_headers`
Tracking visitors	No cookies, no analytics tool, fonts and videos without trackers.	Privacy page

SAY IT

“The best security is having little to protect: a static site has no database and no password to steal. The few secrets are kept in a single place.”

17 · Speed (performance)

A slow site drives people away (and Google penalises it). The recipes used here:

- **Pages already built** (static): nothing to compute during the visit.
- **Images**: WebP, several sizes, lazy loading (chapter 9). The first big photo, however, loads first (`fetchpriority="high"`).
- **Very little JavaScript**: only small scripts where useful. Astro sends no JavaScript by default.
- **Fonts hosted on the site** instead of Google Fonts.
- **Videos loaded on click**.
- **Caching**: files whose name contains a fingerprint (the `_astro/` folder) are kept by the browser for a year (`public/_headers`).

Result measured with **Lighthouse** (built into Chrome, “Lighthouse” tab) on mobile: performance 98–99/100, home page ≈ 256 KB.

The indicators to know

Indicator	Meaning	Target
LCP	Time to show the largest element (often the main photo)	< 2.5 s
CLS	Does the page “jump” while loading?	< 0.1
INP / TBT	Does the page respond quickly to clicks?	as low as possible

18 · Testing the website

Automatic tests

Small programs check that the code does what it should. Here with the tool built into Node (`node:test`), run with `npm test`:

- `tests/calendar.test.ts`: “10 am in Athens in December = 8 am universal time, but 7 am in July (summer time)”, “the .ics file is well formed”…
- `tests/worker.test.ts`: “a wrong password is refused”, “a fake image file is rejected”, “the secret key never reaches the browser”…
- `tests/ai.test.ts`: “only new or changed Greek is translated”, “a translation typed by hand is kept”…

Manual checks (after every big change)

What	How
Display	On a real phone + the developer tools' device mode; Chrome, Safari, Firefox.
Broken links	Click everywhere, or use a link-checking tool.
Accessibility	Lighthouse, the **axe DevTools** extension, keyboard navigation.
Errors	F12 → “Console” tab: no red lines.
Content	Proofread in every language (ideally by native speakers).

19 · Going live: hosting, domain, DNS

Hosting

The `dist/` folder built by `npm run build` is placed with a **host**. For a static site, free services are enough: **GitHub Pages** (used today), **Cloudflare Pages** (planned next), Netlify…

The publishing robot

`.github/workflows/deploy.yml` is a recipe that GitHub runs automatically on every change to `main`: install Node → `npm ci` → `npm run build` → publish `dist/`. This is **continuous integration / deployment** (CI/CD): nobody needs to "upload" anything by hand.

The domain name

A name like `evergreenproject.gr` is bought (≈ €10–15/year) from a **registrar** (Papaki, DNHOST, OVH…). You rent it by the year, you do not own it: watch the renewal!

DNS: the Internet's phone book

DNS turns a name (`evergreenproject.gr`) into a server address. You add "records":

- **A**: "this name points to this server (IP address)".
- **CNAME**: "this name is an alias of that other name" (e.g. `www` → the host).
- **MX**: "e-mails for this domain go here".

> **LOOK**
>
> In `README.md`, the sections on the own domain name and on leaving GitHub: the concrete steps to connect a domain and change host.

20 · The life of a change

For the developer (changing the design or a feature)

1. Create a **branch** (a working copy).
2. Edit the files in VS Code; check live with `npm run dev`.
3. Run the tests (`npm test`) and the build (`npm run build`).
4. **Commit** with a clear message, **push** to GitHub.
5. Open a **pull request**, review, then **merge** into `main`.
6. The robot publishes: ~2 minutes later, everyone can see it.

For an admin (changing a text, a date, a photo)

1. Open `/admin/`, sign in.
2. Edit, click **Save**.
3. ~2 minutes later, it is online. Nothing to install.

REAL EXAMPLE

In the repository's history (“Commits”), look for “Inhabitants: new photo of our dogs”: a new photo added in `src/assets/uploads/` and a single line changed in `src/content/inhabitants/dogs.yml` — exactly what the admin does when a photo is changed.

21 · Where to start learning

A suggested path (about 3 months at a few hours a week)

Step	Goal	Free resources
1. HTML	Make a page with headings, text, images, links	MDN Web Docs (“Learn web development”), freeCodeCamp
2. CSS	Colours, fonts, then Flexbox and Grid	The **Flexbox Froggy** and **Grid Garden** games, web.dev/learn/css
3. Responsive	A page that adapts to phones	web.dev/learn/design
4. Git & GitHub	Saving and publishing your work	GitHub Skills, “Learn Git Branching”
5. JavaScript	Adding a little interaction (button, menu)	javascript.info, freeCodeCamp
6. A generator	Rebuilding a small site with Astro	docs.astro.build (the “Build a blog” tutorial)
7. Publishing	Putting your site online for free	GitHub Pages, Cloudflare Pages

Small exercises on this site (risk-free, on your own computer)

1. Change the `--clay` colour in `src/styles/global.css` and watch.
2. Add a fake event: copy a file in `src/content/events/`, change the dates and the title.
3. Change a button's text in `src/content/settings/ui.yml`.
4. Add an inhabitant (an animal!) in `src/content/inhabitants/`.
5. Read `src/components/Story.astro`, then try to create your own component.

> **FINAL ADVICE**
>
> “You don't learn by reading, you learn by breaking things and fixing them. With Git, nothing is ever really broken: you can always go back.”

22 · Glossary

API	A door that a service opens to other programs.
Build	Producing the final site (the `dist/` folder).
Cache	A copy kept by the browser so it doesn't download again.
CI/CD	Robots that test and publish automatically.
CMS	An interface to manage content without coding (here Sveltia, at `/admin/`).
Commit	A change saved in the history, with a message.
Component	A reusable page brick (`Img`, `Story`…).
CSP	A security whitelist of what a page may load.
DNS	The phone book that turns a domain name into a server address.
Repository (repo)	The project folder with its whole history (on GitHub).
Front-end / back-end	What runs on the visitor's side / on the server.
Host	A service that keeps the site's files online 24/7.
i18n	“Internationalisation” (i + 18 letters + n): handling languages.
Layout	The frame shared by all pages.
Markdown	A simple way to write formatted text (`**bold**`).
npm / package	The JavaScript tool shop / a tool downloaded from it.
Pull request	A request to merge a branch, with review.
Responsive	Adapting to every screen size.
SEO	Being found and presented well by Google.
SRI	A fingerprint guaranteeing an external file has not been changed.
Static	Pages built in advance rather than on every visit.
URL	The address of a page.
WebP	A modern image format, lighter than JPEG.
Worker	A small server function hosted at Cloudflare.
YAML	A readable text format (“key: value”) that stores the content.